Ders 1 Çalışma Özeti

Ders 1 Çalışma Özeti

Genel Konular

- Dersin kapsamı ve kaynakları (ekrandan, 00:10:00 ve 00:10:40)
  - Ders iki kısımdan oluşur: Kısım 1 Gang of Four (Dörtlü Çete) tasarım kalıpları, Kısım 2 Code Smells and Refactoring (birim sınamaları ile).
  - Kısım 1'in temel kaynağı: *Design Patterns: Elements of Reusable Object-Oriented Software*, Erich Gamma vd. (Gang of Four), Addison-Wesley, 1994. Literatürdeki klasik tasarım kalıpları bu kitapta tanımlanmıştır.
  - Ek kaynaklar (slayttaki liste): *Design Patterns Explained* (Shalloway, Trott), *Head First Design Patterns* (Freeman vd.), *Design Patterns* (Lasater), *Design Patterns Java Workbook* (Metsker), *Applied Java Patterns* (Stelting, Maassen). Bu kitaplar klasik kalıpları bütüncül bir proje örneği (örneğin bir öğrenci bilgi sistemi) üzerinden ve birden fazla kalıbı birlikte kullanarak anlatır.
  - Kısım 2'nin kaynağı: *Refactoring: Improving the Design of Existing Code*, Martin Fowler, Addison-Wesley, 1999.
  - Dersin çekirdeği: nesneye dayalı tasarım ve modellemeyi (a) nasıl iyi yapacağımız, (b) kötü yapılmışsa nasıl düzelteceğimiz.
- Ön koşullar: NYP dili ve UML sınıf şemaları (ekrandan, 00:10:40)
  - Herhangi bir güncel nesneye yönelik programlama diline hakimiyet gerekir. Kod örnekleri Java ile verilir, ama Java'ya özgü bir özellik kullanılmaz; kod C#'a (package yerine namespace) ya da küçük notasyon farkıyla C++'a aktarılabilir.
  - UML (Unified Modeling Language), analiz ve tasarımın teknik belgelendirilmesi için sektördeki standarttır ve birçok şema türü içerir. Bu derste tasarımlar UML sınıf şemalarıyla gösterilir.
  - Önerilen UML kaynakları: *UML Distilled* (Martin Fowler, 3rd ed., 2003) ve *UML 2.0 in a Nutshell* (Pilone, Pitman, O'Reilly). UML Distilled her şema öğesini anlatırken arkasındaki kavramı da (örneğin kalıtımın anlamı, çok biçimlilikle kullanımı, ne zaman doğru ne zaman yanlış olduğu) anlatır.
- Yazılım kalitesi (ekrandan, 00:40:00)
  - Amaç "iyi" tasarım ile "kaliteli" yazılıma ulaşmaktır. Hocanın sevdiği tanım: problem alanı hangi düzeyde parçalara ayrılabiliyorsa, o düzeyde parçalara ayrılabilen bir tasarım iyidir. Bu tanım objektif ölçütlere dayanmadığı için daha somut ölçütler gerekir.
  - Dış kalite özellikleri kullanıcıya yöneliktir: doğruluk, etkinlik, kolaylık vb.
  - İç kalite özellikleri programcıya yöneliktir: yeniden kullanılabilirlik, ilgi alanlarının ayrılması, esneklik, taşınabilirlik, okunabilirlik ve anlaşılabilirlik, sınanabilirlik, bakım kolaylığı vb. Tasarım kalıpları da refactoring de iç kalite özelliklerini iyileştirmeye yarar.
- Yazılım projelerinde başarı oranları (ekrandan, 00:42:40 ve 00:43:45)
  - Bir proje önceden belirlenmiş zamanda, bütçeyle ve gereksinimleri karşılayarak bitmişse başarılıdır. Birinde bile olumsuz sapma varsa başarısız sayılır.
  - Gösterilen kaynak: Dobb's Journal 2013 IT Project Success Survey. Bu sıkı tanımla geleneksel (şelale) süreçlerde başarı yaklaşık %50, çevik süreçlerde yaklaşık %65, Lean'de %70'ler civarındadır.
  - "Zorlanılmış" (bütçe, süre, işlev yönlerinden en az birinde başarısızlık) projeler de sayılırsa oran geleneksel süreçte %80-82'ye (grafikte geleneksel: zorlanılmış %32, başarısız %18), çevik süreçte %90'a çıkar.

- Bağlaşım (coupling) ve uyum (cohesion)
  - İyi tasarımda düşük bağlaşım (low coupling) ve yüksek uyum (high cohesion) olmalıdır. Genelde biri diğerini beraberinde getirir.
  - Bağlaşım: bir parçanın diğer parçalarla ilişkide bulunma oranı. Sınıf, alt sistem ya da paket düzeyinde düşünülebilir; derste sınıf düzeyi ele alınır. Bir sınıf diğerinin hizmetlerinden (metotlarından) yararlanıyorsa, iç yapısından haberdarsa ya da aralarında kalıtım varsa bağlaşım vardır. İlişkide bulunulan sınıf sayısı arttıkça bağlaşım artar.
  - Düşük bağlaşımda bir sınıftaki değişikliğin diğer sınıfları olumsuz etkileme olasılığı düşer. Sıfır bağlaşım olamaz; önemli olan gereksiz ilişki kurmamaktır.
  - Uyum: bir parçanın sorumluluklarının birbirleriyle uyumlu olma oranı. Uyum yükseldikçe sınıf daha kolay anlaşılır, yeniden kullanılabilirlik ve bakım kolaylığı artar, sınıfın değişikliklerden etkilenme olasılığı düşer.
- Bağlaşım ölçme örneği (ekrandan, 00:52:40 ve 01:00:00)
  - Tasarım I: A → B → C (A'da B türünden private üye alan `- b : B`, B'de C türünden). Tasarım II: A → B ve A → C.
  - Doğrudan ilişkiler sayılırsa: Bağlaşım(I) = 2 (Bağ(A) = 1, Bağ(B) = 1, Bağ(C) = 0, en kötü durum 1). Bağlaşım(II) = 2 (Bağ(A) = 2, Bağ(B) = 0, Bağ(C) = 0, en kötü durum 2). En kötü duruma göre Tasarım I daha iyidir.
  - Dolaylı A → C ilişkisi yarım sayılırsa: Bağ(A) = 1,5 ve Bağlaşım(I) = 2,5; Tasarım II'de dolaylı ilişki yoktur, 2 kalır. Bu ölçüyle Tasarım II daha iyidir.
  - Sonuç: sayılarla çalışsak da ölçütün nasıl tanımlandığı sonucu değiştirir. Ayrıca A, B, C gerçek dünyadaki varlıklara karşılık gelir; gerçek dünyadaki ilişki Tasarım I gibiyse o seçilir.
- Uyum örneği: veritabanı sorumluluğu (ekrandan, 01:05:40, 01:10:00 ve 01:11:40)
  - Seçenek 1: A, B, C kendilerini veritabanına kaydeder ve yükler (her biri VT'ye bağımlı). Seçenek 2: nesneleri veritabanına yazmaktan ve okumaktan sorumlu ayrı bir D sınıfı; D, A, B, C ile en azından kullanma (dependency, kesikli ok) ilişkisi kurar.
  - Kullanma ilişkileri yarım sayılırsa Bağ(II) = 2 + 3 × 0,5 = 3,5 olur; bağlaşım artar ama uyum yükselir (tek sorumluluk, ilgi alanlarının ayrılması).
  - Veritabanı MySQL'den PostgreSQL'e ya da ilişkisel olmayan bir sisteme geçerse yalnız D değişir; A, B, C altyapıdan bağımsız kalır. Bu, iş mantığı katmanı ile veritabanı (kalıcılık) katmanının ayrılması, yani katmanlama sağlar.
- Tasarım kalıbı nedir? (ekrandan, 01:20:00)
  - Yazılım tasarımında karşılaşılan belirli sorunların yalın ve güzel çözümlerinin tarifidir. Çözüm somut bir gerçekleme olmak zorunda değildir, soyut bir tarif de olabilir.
  - Amaç: tekerleği yeniden keşfetmeden "iyi" tasarıma ulaşmak; sorunları esnek ve yeniden kullanılabilirliği artıracak yönde çözmek.
  - Kavram mimarlıktan ödünç alınmıştır (örnek: içinde insanların uzun süre bulunacağı her odada doğal aydınlatma, pencere olmalı).
  - Dört ana bileşen: İsim (kalıbı en iyi anlatan bir ya da birkaç kelime), Sorun (çözülen sorunun tarifi), Çözüm (bileşenler, ilişkileri, sorumlulukları, birlikte çalışmaları), Tartışma/Sonuçlar (esneklik, genişletilebilirlik, taşınabilirlik üzerindeki etkiler; güçlü ve zayıf yönler; yer-zaman çelişkileri; başka kalıplarla işbirliği olanakları).
- GoF kalıplarının ilişki şeması ve sınıflandırması (ekrandan, 01:21:50, 01:27:20 ve 01:30:00)
  - GoF kitabında kalıplar arası işbirliğini oklarla gösteren bir şema vardır (örneğin Strategy → Flyweight "sharing strategies"); ezberlemek gerekmez. Derste Interpreter dışındaki bütün GoF kalıpları işlenecektir.
  - Amaçlarına göre üç küme: Creational (yaratımsal; nesnelerin oluşturulması, temsili ve ilişkilendirilmesindeki bağımlılığı azaltma), Structural (yapısal; sınıf ve nesneleri bir araya getirerek daha büyük yapılar elde etme), Behavioral (davranışsal; sorumlulukların nesnelere dağıtılması ve algoritma seçimi). Ders notları kitaptaki sıralamaya sadık kalır.
  - Odağa göre: sınıfsal kalıplar (sınıflar ve alt sınıfları arası ilişkiler; derleme zamanında belirlenen, durağan ilişkiler) ve nesnesel kalıplar (çeşitli türden nesneler arası ilişkiler; çalışma süresince değiştirilebilen, dinamik ilişkiler). Adapter hem sınıfsal hem nesnesel sayılır; diğerleri yalnız birine girer.
- Örnek tasarım kalıbı: Model-View-Controller (ekrandan, 01:31:45, 01:35:10, 01:41:15 ve 01:45:25)

  - Sorun: aynı veriyi farklı biçimlerde göstermek; kullanıcıya verinin nasıl ve ne kadarının gösterileceğini seçme olanağı vermek.
  - Çözüm: veriyi, verinin gösterimini ve gösterimin yönetimini ayrı sınıflar olarak ele almak. Model ham veriyi, View verinin çeşitli gösterim biçimlerini simgeler; Controller kullanıcı komutlarını alıp işler.
  - Temel yapı: Controller → View, Controller → Model, View → Model (toplam bağlaşım 3).
  - Örnek veri: bir seçimde A partisi %40, B %30, C %10, diğer %20; çubuk ya da pasta grafik, ilk n parti gibi farklı gösterimler.
  - Model değişince View'un haberdar olması için ya View belli aralıklarla modeli yoklar (aralık kısa olursa gereksiz yük) ya da Model → View ilişkisi eklenir (bağlaşım 4 olur). Kullanıcı değişikliği View üzerinden giriyorsa (`updateSomething()`), diğer View'ların `refresh()` ile yenilenmesi için View → Controller ilişkisi de gerekebilir. Controller birden fazla View ile ilişkili olabilir (`1..*`).
  - Yeni bir giriş yolu (ses ile denetim) eklenecekse Controller'dan kalıtımla `SesleDenetim` sınıfı türetilebilir.

Hocanın Özellikle Vurguladığı Kısımlar

- UML sınıf şemalarına hakimiyet şart
  - Nesneye dayalı tasarımı göstermenin en yaygın, neredeyse tek yolu UML sınıf şemalarıdır. Eksik olanlar mutlaka UML çalışmalı; hoca bu konuda eksik öğrencilerin çok olduğunu söyledi.
  - NYP dili ile UML'in ayrı ayrı değil, bütüncül öğretilmesinden yanadır. Lisans NYP ders notlarında koda yakın sınıf şemaları ve sequence (sıralama) şemaları birlikte anlatılır; temeli bayatlamış olanlara bu notları gözden geçirmeleri önerildi.
- Bakım kolaylığı ve erken hata bulma
  - Bir yazılım projesindeki çabanın, emeğin ve maliyetin rahatça yarısı bakım aşamasında harcanır. Okunabilirlik ve anlaşılabilirlik bakım kolaylığını doğrudan etkiler.
  - Bir hatayı düzeltmenin bedeli yazılım yaşam döngüsünde ilerledikçe üstel artar; şelale süreciyle çalışılıyorsa en baştan iyi tasarım yapmak zorunludur.
- UML'de okun yönü önemlidir
  - A'dan B'ye ok varsa A, B'ye bağımlıdır; B, A'ya bağımlı değildir. A'daki değişiklik B'yi etkilemez.
  - Düz ok (association): A sınıfında B türünden (en az bir) private üye alan vardır. Kesikli ok (kullanma, dependency): üye alan olmadan, bir metot içinde geçici değişken ya da metot parametresi olarak kullanma.
- Bağlaşım ve uyum genelde birlikte gider, ama istisnalar vardır
  - Veritabanı örneği istisnai bir örnektir: bağlaşım kötüleşir ama uyumu daha yüksek tasarım seçilir.
- Tasarım kalıbı dogma değildir
  - Emir ya da yasak değil, kendi ihtiyacınıza göre değiştirebileceğiniz bir çözüm önerisidir. View'u hafif tutmak gerekiyorsa bağlaşımı artırmak bir yatırım olarak görülebilir; gerek yoksa gereksiz ilişki eklenmez.
  - Kalıp doğru uygulanmazsa ya da "illa bu kalıbı uygulayacağız" diye düşünülürse esnekliği azaltan bir iş yapılmış olabilir.
- Tasarım kalıpları ortak sözlük sağlar
  - Ekip içinde "her bölgenin vergi kuralları birer strateji olsun", "bazı stratejiler aynı zamanda Builder ile kurulsun" gibi kısa cümleler 10-12 sınıflık bir tasarımı anlatabilir.

Kısa Tekrar Notları

- Ders: GoF tasarım kalıpları (Interpreter hariç hepsi) ve Fowler'ın code smells/refactoring konusu.
- İyi tasarım: problem alanı hangi düzeyde parçalanabiliyorsa o düzeyde parçalanan tasarım.
- Dış kalite kullanıcıya (doğruluk, etkinlik, kolaylık), iç kalite programcıya (yeniden kullanılabilirlik, ilgi alanlarının ayrılması, esneklik, taşınabilirlik, okunabilirlik, sınanabilirlik,

bakım kolaylığı) yöneliktir.
- Başarılı proje: zamanında, bütçesinde, gereksinimleri karşılayarak biten proje.
- Düşük bağlaşım + yüksek uyum = iyi tasarım. Bağlaşım ilişkide olunan sınıf sayısıyla artar.
- Bağlaşım değeri ölçütün tanımına bağlıdır (doğrudan / dolaylı ilişki, en kötü durum).
- Kalıcılık sorumluluğunu ayrı bir sınıfa vermek uyumu artırır ve katmanlamayı sağlar.
- Tasarım kalıbının dört bileşeni: isim, sorun, çözüm, tartışma/sonuçlar.
- GoF kümeleri: yaratımsal, yapısal, davranışsal. Odak: sınıfsal ya da nesnesel; Adapter ikisi birden.
- MVC: Model = veri, View = gösterim, Controller = kullanıcı komutları ve yönetim.
- UML: düz ok üye alan (association), kesikli ok kullanma (dependency), içi boş üçgen kalıtım, * sıfır ya da daha fazla, `1..*` bir ya da daha fazla.

Detaylı Açıklamalar

- Tasarım kalıpları ve refactoring neden birlikte?
  - Tasarım kalıpları nasıl iyi tasarım yapılacağını söyler. Code smells ise bug olmayan ama esnek olmayan, en küçük hata düzeltmesinde ya da gereksinim değişikliğinde gereğinden fazla iş çıkaran modellemelerdir. Bunları tanıyıp kaçınmak ve karşılaşıldığında düzeltmek refactoring'dir. Hoca Fowler'ın kitabındaki code smell'lerin yaklaşık yarısını, sektörde en çok karşılaşılanlar olarak seçip notlarına almıştır.
  - Kalıpların eskiliği yanıltmamalıdır: çözüm önerileri nesneye dayalı modellemenin çekirdeği (parça-bütün ilişkileri, kalıtım vb.) üzerinedir ve bu çekirdek ilk önerildiği 1960'lardan beri pek değişmemiştir.
- Violet UML Editor
  - Hoca sınıf şemalarını Violet UML Editor ile çizer (ekrandan, 00:50:00: `VioletUML-0.21.1`, küçük bir çalıştırılabilir jar dosyası). Sınıf, use case, state, activity, sequence şemaları sürükle-bırak çizilir, saklanır ve PNG olarak dışa aktarılır; kod üretme yeteneği yoktur. Çalışması için Java Runtime Environment gerekir. Yeni sürümü eski sürümün dosyalarıyla uyumsuz olduğu için hoca eski sürümü kullanır; öğrenciler yeni sürümü de kullanabilir.
- Bağlaşım örneğinde öğrenci yorumları
  - Sınıf oylamasında (ekrandan, 01:00:00) Tasarım I'e 2, Tasarım II'ye 8 oy geldi. Bir öğrenci "C'de sorun çıkarsa Tasarım I'de hem A hem B etkilenir, Tasarım II'de B etkilenmez" diye Tasarım II'yi savundu. Hoca bunu mantıklı buldu; dolaylı ilişkiler sayıldığında gerçekten Tasarım II öne geçer.
  - Bir başka öğrenci "B çıkarılırsa Tasarım I'de A ve C etkilenir" dedi; hoca okun yönünü düzeltti: A → B okunda A, B'ye bağımlıdır, B'deki değişiklik A'yı etkiler ama A'daki değişiklik B'yi etkilemez.
- Veritabanı örneğinde gerekçe
  - Sınıfın tamamı Tasarım II'yi (ayrı D sınıfı) seçti. Öğrenciler gerekçe olarak single responsibility ilkesini ve karmaşıklığın azalmasını söyledi. Hoca bunu ilgi alanlarının ayrılması ve uyum açısından doğruladı; uzamsal ya da graf verisi gerekirse ilişkisel olmayan bir veritabanına geçilebileceğini, o zaman yalnız D'nin değişmesinin büyük kazanç olduğunu anlattı.
- MVC'de ressam benzetmesi
  - Model, ressamın resmini çizdiği kişidir (veri). Controller ressamdır: kişiyi nasıl ve ne kadar çizeceğine o karar verir. View ortaya çıkan resimdir: tüm boy ya da omuzdan yukarısı (portre), gerçekçi, karikatür ya da sürrealist biçimde.
  - View'lar iç içe (nested) olabilir, çok iş parçacıklı (multi-threaded) çalışabilir; birden fazla Controller (klavye, fare, ses) olabilir. Kalıp bunlar için kesin bir şey dayatmaz.
  - Aynı veriye birden fazla View bakıyorsa ileride görülecek Observer kalıbı devreye girebilir.
  - Ders sonrası not: Hoca bu kalıbı "Observable" diye andı; GoF kitabındaki adı Observer'dır. Hoca MVC'yi "Dörtlü Çete'nin kalıbı" diye tanıttı; MVC büyük olasılıkla GoF'un 23 kalıbı arasında değildir, kitabın giriş bölümünde Smalltalk MVC örneği olarak anlatılır.
- **Not:** İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.